

The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Development Kit

Mark Everingham John Winn

May 18, 2012

Contents

1	Challenge	4
2	Data	4
2.1	Classification/Detection Image Sets	5
2.2	Segmentation Image Sets	5
2.3	Action Classification Image Sets	5
2.4	Person Layout Taster Image Sets	8
2.5	Ground Truth Annotation	9
2.6	Segmentation Ground Truth	9
2.7	Person Layout Taster Ground Truth	10
2.8	Action Classification Ground Truth	10
3	Classification Task	11
3.1	Task	11
3.2	Competitions	11
3.3	Submission of Results	11
3.4	Evaluation	12
3.4.1	Average Precision (AP)	12
4	Detection Task	12
4.1	Task	12
4.2	Competitions	12
4.3	Submission of Results	13
4.4	Evaluation	13
5	Segmentation Task	14
5.1	Task	14
5.2	Competitions	14
5.3	Submission of Results	14
5.4	Evaluation	15

6	Action Classification Task	15
6.1	Task	15
6.2	Competitions	15
6.3	Submission of Results	16
6.4	Evaluation	16
7	Boxless Action Classification Taster	16
7.1	Task	16
7.2	Competitions	17
7.3	Submission of Results and Evaluation	17
8	Person Layout Taster	17
8.1	Task	17
8.2	Competitions	18
8.3	Submission of Results	18
8.4	Evaluation	19
9	Development Kit	20
9.1	Installation and Configuration	20
9.2	Example Code	21
9.2.1	Example Classifier Implementation	21
9.2.2	Example Detector Implementation	21
9.2.3	Example Segmenter Implementation	21
9.2.4	Example Action Implementation	22
9.2.5	Example Boxless Action Implementation	22
9.2.6	Example Layout Implementation	22
9.3	Non-MATLAB Users	22
10	Using the Development Kit	22
10.1	Image Sets	22
10.1.1	Classification/Detection Task Image Sets	22
10.1.2	Classification Task Image Sets	23
10.1.3	Segmentation Image Sets	24
10.1.4	Action Classification Image Sets	24
10.1.5	Action Class Image Sets	24
10.1.6	Person Layout Taster Image Sets	25
10.2	Development Kit Functions	26
10.2.1	VOCinit	26
10.2.2	PASreadrecord(filename)	26
10.2.3	viewanno(imgset)	29
10.3	Classification Functions	30
10.3.1	VOCevalcls(VOCopts,id,cls,draw)	30
10.4	Detection Functions	30
10.4.1	VOCevaldet(VOCopts,id,cls,draw)	30
10.4.2	viewdet(id,cls,onlytp)	30
10.5	Segmentation Functions	30
10.5.1	create_segmentations_from_detections(id,confidence)	30
10.5.2	VOCevalseg(VOCopts,id)	31
10.5.3	VOClabelcolormap(N)	31
10.6	Action Functions	31

10.6.1	VOCevalaction(VOCopts,id,cls,draw)	31
10.7	Layout Functions	31
10.7.1	VOCwritexml(rec,path)	31
10.7.2	VOCevallayout_pr(VOCopts,id,draw)	32

1 Challenge

The goal of this challenge is to recognize objects from a number of visual object classes in realistic scenes (i.e. not pre-segmented objects). There are twenty object classes:

- person
- bird, cat, cow, dog, horse, sheep
- aeroplane, bicycle, boat, bus, car, motorbike, train
- bottle, chair, dining table, potted plant, sofa, tv/monitor

There are five main tasks:

- Classification: For each of the classes predict the presence/absence of at least one object of that class in a test image.
- Detection: For each of the classes predict the bounding boxes of each object of that class in a test image (if any).
- Segmentation: For each pixel in a test image, predict the class of the object containing that pixel or ‘background’ if the pixel does not belong to one of the twenty specified classes.
- Action Classification: For each of the action classes predict if a specified person (indicated by their bounding box) in a test image is performing the corresponding action. There are ten action classes:
 - jumping; phoning; playing a musical instrument; reading; riding a bicycle or motorcycle; riding a horse; running; taking a photograph; using a computer; walking

In addition, some people are performing the action “other” (none of the above) and act as distractors.

- Large Scale Recognition: This task is run by the ImageNet organizers. Further details can be found at their website: <http://www.image-net.org/challenges/LSVRC/2012/index>.

In addition, there are two “taster” tasks:

- Boxless Action Classification: For each of the action classes predict if a specified person in a test image is performing the corresponding action. The person is indicated only by a single point lying somewhere on their body, rather than by a tight bounding box.
- Person Layout: For each ‘person’ object in a test image (indicated by a bounding box of the person), predict the presence/absence of parts (head/hands/feet), and the bounding boxes of those parts.

2 Data

The VOC2012 data is released in two phases: (i) training and validation data with annotation is released with this development kit; (ii) test data without annotation is released at a later date.

2.1 Classification/Detection Image Sets

For the classification and detection tasks there are four sets of images provided:

train: Training data

val: Validation data (suggested). The validation data may be used as additional training data (see below).

trainval: The union of **train** and **val**.

test: Test data. The test set is not provided in the development kit. It will be released in good time before the deadline for submission of results.

Table 1 summarizes the number of objects and images (containing at least one object of a given class) for each class and image set. The data has been split into 50% for training/validation and 50% for testing. The distributions of images and objects by class are approximately equal across the training/validation and test sets.

Note that the 2012 data for the main classification/detection tasks is the *same* as the 2011 data – no additional images have been annotated. The assignment of images to training/test sets has not been changed. The dataset includes images from the 2008–2011 datasets, for which no test set annotation has been released.

2.2 Segmentation Image Sets

For the segmentation task, corresponding image sets are provided as in the classification/detection tasks. To increase the amount of data, the training and validation image sets include images from the 2007–2011 segmentation tasks. The test set contains only 2008–2011 images (i.e. those for which no annotation has been released), and is a subset of the test set for the main tasks for which pixel-wise segmentations have been prepared. Table 2 summarizes the number of objects and images (containing at least one object of a given class) for each class and image set, for the combined 2007–2012 data. In addition to the segmented images for training and validation, participants are free to use the un-segmented training/validation images supplied for the main classification/detection tasks, and any annotation provided for the main challenge e.g. bounding boxes.

2.3 Action Classification Image Sets

For the action classification task, corresponding image sets are provided as in the classification/detection tasks. The same images are used for both the action classification task (where a person is indicated by a bounding box) and the boxless action classification task (where a person is indicated by a point lying somewhere on the body). Each person has been annotated with the set of actions they are performing from the set {jumping, phoning, playing a musical instrument, reading, riding a bicycle or motorcycle, riding a horse, running, taking a photograph, using a computer, walking}. Note that actions are not mutually exclusive, for example a person may simultaneously be walking and phoning. Additionally, there are people labeled with the action ‘other’, meaning none of the above actions are being performed. These examples make serve as distractors to the main ten action classes.

Table 1: Statistics of the main image sets. Object statistics list only the ‘non-difficult’ objects used in the evaluation.

	train		val		trainval		test	
	img	obj	img	obj	img	obj	img	obj
Aeroplane	327	432	343	433	670	865	–	–
Bicycle	268	353	284	358	552	711	–	–
Bird	395	560	370	559	765	1119	–	–
Boat	260	426	248	424	508	850	–	–
Bottle	365	629	341	630	706	1259	–	–
Bus	213	292	208	301	421	593	–	–
Car	590	1013	571	1004	1161	2017	–	–
Cat	539	605	541	612	1080	1217	–	–
Chair	566	1178	553	1176	1119	2354	–	–
Cow	151	290	152	298	303	588	–	–
Diningtable	269	304	269	305	538	609	–	–
Dog	632	756	654	759	1286	1515	–	–
Horse	237	350	245	360	482	710	–	–
Motorbike	265	357	261	356	526	713	–	–
Person	1994	4194	2093	4372	4087	8566	–	–
Pottedplant	269	484	258	489	527	973	–	–
Sheep	171	400	154	413	325	813	–	–
Sofa	257	281	250	285	507	566	–	–
Train	273	313	271	315	544	628	–	–
Tvmonitor	290	392	285	392	575	784	–	–
Total	5717	13609	5823	13841	11540	27450	–	–

Table 2: Statistics of the segmentation image sets.

	train		val		trainval		test	
	img	obj	img	obj	img	obj	img	obj
Aeroplane	88	108	90	110	178	218	—	—
Bicycle	65	94	79	103	144	197	—	—
Bird	105	137	103	140	208	277	—	—
Boat	78	124	72	108	150	232	—	—
Bottle	87	195	96	162	183	357	—	—
Bus	78	121	74	116	152	237	—	—
Car	128	209	127	249	255	458	—	—
Cat	131	154	119	132	250	286	—	—
Chair	148	303	123	245	271	548	—	—
Cow	64	152	71	132	135	284	—	—
Diningtable	82	86	75	82	157	168	—	—
Dog	121	149	128	150	249	299	—	—
Horse	68	100	79	104	147	204	—	—
Motorbike	81	101	76	103	157	204	—	—
Person	442	868	445	865	887	1733	—	—
Pottedplant	82	151	85	171	167	322	—	—
Sheep	63	155	57	153	120	308	—	—
Sofa	93	103	90	106	183	209	—	—
Train	83	96	84	93	167	189	—	—
Tvmonitor	84	101	74	98	158	199	—	—
Total	1464	3507	1449	3422	2913	6929	—	—

Table 3: Statistics of the action classification image sets.

	train		val		trainval		test	
	img	obj	img	obj	img	obj	img	obj
Jumping	204	247	201	248	405	495	–	–
Phoning	223	228	221	229	444	457	–	–
Playinginstrument	230	309	229	310	459	619	–	–
Reading	232	264	231	266	463	530	–	–
Ridingbike	201	289	199	289	400	578	–	–
Ridinghorse	205	266	206	268	411	534	–	–
Running	156	280	154	281	310	561	–	–
Takingphoto	209	227	205	229	414	456	–	–
Usingcomputer	199	237	196	239	395	476	–	–
Walking	191	298	195	299	386	597	–	–
Other	402	521	397	522	799	1043	–	–
Total	2296	3134	2292	3144	4588	6278	–	–

Table 4: Statistics of the person layout taster image sets. Object statistics list only the ‘person’ objects for which layout information (parts) is present.

	train		val		trainval		test	
	img	obj	img	obj	img	obj	img	obj
Person	315	425	294	425	609	850	–	–

The image sets are disjoint from those of the classification/detection tasks and person layout taster task. Note that they are *not* fully annotated – only ‘person’ objects forming part of the training and test sets are annotated, and there may be unannotated people in the images. Table 3 summarizes the action statistics for each image set.

2.4 Person Layout Taster Image Sets

For the person layout taster task, corresponding image sets are provided as in the classification/detection tasks. A person is indicated by a bounding box, and each person has been annotated with part layout (head, hands, feet). Note that the 2012 person layout dataset is the *same* as the corresponding 2011 dataset – no additional images have been annotated. The assignment of images to training/test sets has not been changed. The dataset includes images from the 2008–2011 datasets, for which no test set annotation has been released, and the test set is disjoint from that for the main classification/detection tasks. Table 4 summarizes the number of ‘person’ objects annotated with layout for each image set.

Note that the person layout images in VOC2012 are *not* fully annotated – only a *subset* of people have been annotated, and *no* other objects – for example an image containing two people and a car may only have one person annotated.

2.5 Ground Truth Annotation

Objects of the twenty classes listed above are annotated in the ground truth. For each object, the following annotation is present:

- **class**: the object class e.g. ‘car’ or ‘bicycle’
- **bounding box**: an axis-aligned rectangle specifying the extent of the object visible in the image.
- **view**: ‘frontal’, ‘rear’, ‘left’ or ‘right’. The views are subjectively marked to indicate the view of the ‘bulk’ of the object. Some objects have no view specified.
- **‘truncated’**: an object marked as ‘truncated’ indicates that the bounding box specified for the object does not correspond to the full extent of the object e.g. an image of a person from the waist up, or a view of a car extending outside the image.
- **‘occluded’**: an object marked as ‘occluded’ indicates that a significant portion of the object within the bounding box is occluded by another object.
- **‘difficult’**: an object marked as ‘difficult’ indicates that the object is considered difficult to recognize, for example an object which is clearly visible but unidentifiable without substantial use of context. Objects marked as difficult are currently *ignored* in the evaluation of the challenge.

In preparing the ground truth, annotators were given a detailed list of guidelines on how to complete the annotation. These are available on the main challenge web-site [1].

Note that for the action classification images, only people have been annotated, and only the bounding box and a reference point on the person is available. Note also that for these images the annotation is not necessarily complete i.e. there may be unannotated people.

2.6 Segmentation Ground Truth

For the segmentation image sets, each image has two corresponding types of ground truth segmentation provided:

- **class segmentation**: each pixel is labelled with the ground truth class or background.
- **object segmentation**: each pixel is labelled with an object number (from which the class can be obtained) or background.

Figure 1 gives an example of these two types of segmentation for one of the training set images. The ground truth segmentations are provided to a high degree of accuracy, but are not pixel accurate, as this would have greatly extended the time required to gather these segmentations. Instead, they were labelled so that a bordering region with a width of five pixels may contain either object or background. Bordering regions are marked with a ‘void’ label (index 255), indicating that the contained pixels can be any class including background. The

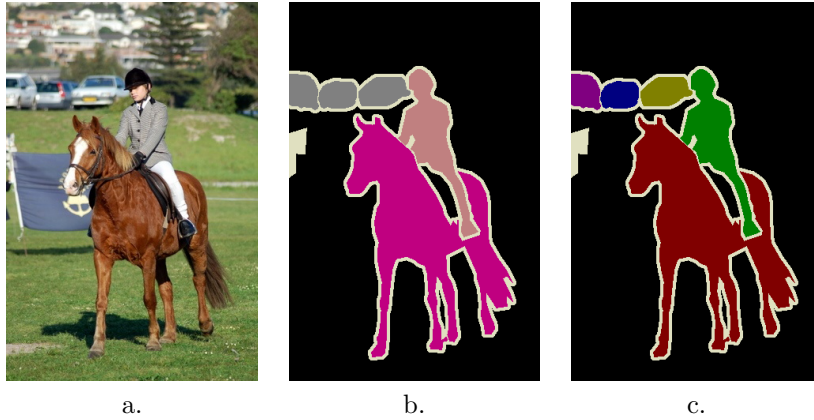


Figure 1: Example of segmentation ground truth. **a.** Training image **b.** Class segmentation showing background, car, horse and person labels. The cream-colored ‘void’ label is also used in border regions and to mask difficult objects. **c.** Object segmentation where individual object instances are separately labelled.

void label is also used to mask out ambiguous, difficult or heavily occluded objects and also to label regions of the image containing objects too small to be marked, such as crowds of people. All void pixels are ignored when computing segmentation accuracies and should be treated as unlabelled pixels during training.

In addition to the ground truth segmentations given, participants are free to use any of the ground truth annotation for the classification/detection tasks e.g. bounding boxes.

2.7 Person Layout Taster Ground Truth

For the person layout taster task, ‘person’ objects are additionally annotated with three ‘parts’:

- head – one per person
- hand – zero, one, or two per person
- foot – zero, one, or two per person

For each annotated person, the presence or absence of each part is listed, and for each part present, the bounding box is specified. The test images for the person layout taster are disjoint from the other image sets. There are no ‘difficult’ objects. As noted above, the layout taster ground truth is incomplete – only people are annotated, and there may be unannotated people in an image.

2.8 Action Classification Ground Truth

For the action classification task, ‘person’ objects are annotated with bounding box, a reference point lying somewhere on the body, and a set of flags, one per action class e.g. ‘phoning’ or ‘walking’. For each action the flag indicates if

the person is performing the corresponding action. Note that actions are not mutually exclusive, for example a person may simultaneously be walking and phoning. The ‘other’ action is mutually exclusive to all other actions. The image sets are disjoint from the classification/detection and layout taster tasks. There are no ‘difficult’ objects.

3 Classification Task

3.1 Task

For each of the twenty object classes predict the presence/absence of at least one object of that class in a test image. The output from your system should be a real-valued confidence of the object’s presence so that a precision/recall curve can be drawn. Participants may choose to tackle all, or any subset of object classes, for example “cars only” or “motorbikes and cars”.

3.2 Competitions

Two competitions are defined according to the choice of training data: (i) taken from the VOC **trainval** data provided, or (ii) from any source excluding the VOC **test** data provided:

No.	Task	Training data	Test data
1	Classification	trainval	test
2	Classification	any but VOC test	test

In competition 1, any annotation provided in the VOC **train** and **val** sets may be used for training, for example bounding boxes or particular views e.g. ‘frontal’ or ‘left’. Participants are *not* permitted to perform additional manual annotation of either training or test data.

In competition 2, any source of training data may be used *except* the provided **test** images. Researchers who have pre-built systems trained on other data are particularly encouraged to participate. The test data includes images from “flickr” (www.flickr.com); this source of images may *not* be used for training. Participants who have acquired images from flickr for training must submit them to the organizers to check for overlap with the test set.

3.3 Submission of Results

A separate text file of results should be generated for each competition (1 or 2) and each class e.g. ‘car’. Each line should contain a single identifier and the confidence output by the classifier, separated by a space, for example:

```
comp1_cls_test_car.txt:
...
2009_000001 0.056313
2009_000002 0.127031
2009_000009 0.287153
...
```

Greater confidence values signify greater confidence that the image contains an object of the class of interest. The example classifier implementation (section 9.2.1) includes code for generating a results file in the required format.

3.4 Evaluation

The classification task will be judged by the precision/recall curve. The principal quantitative measure used will be the average precision (AP). Example code for computing the precision/recall and AP measure is provided in the development kit. See also section 3.4.1.

Images which contain only objects marked as ‘difficult’ (section 2.5) are currently *ignored* by the evaluation. The final evaluation may include separate results including such “difficult” images, depending on the submitted results.

Participants are expected to submit a *single* set of results per method employed. Participants who have investigated several algorithms may submit one result per method. Changes in algorithm parameters do *not* constitute a different method – all parameter tuning must be conducted using the training and validation data alone.

3.4.1 Average Precision (AP)

The computation of the average precision (AP) measure was changed in 2010 to improve precision and ability to measure differences between methods with low AP. It is computed as follows:

1. Compute a version of the measured precision/recall curve with precision monotonically decreasing, by setting the precision for recall r to the maximum precision obtained for any recall $r' \geq r$.
2. Compute the AP as the area under this curve by numerical integration. No approximation is involved since the curve is piecewise constant.

Note that prior to 2010 the AP is computed by sampling the monotonically decreasing curve at a fixed set of uniformly-spaced recall values $0, 0.1, 0.2, \dots, 1$. By contrast, VOC2010–2012 effectively samples the curve at *all* unique recall values.

4 Detection Task

4.1 Task

For each of the twenty classes predict the bounding boxes of each object of that class in a test image (if any). Each bounding box should be output with an associated real-valued confidence of the detection so that a precision/recall curve can be drawn. Participants may choose to tackle all, or any subset of object classes, for example “cars only” or “motorbikes and cars”.

4.2 Competitions

Two competitions are defined according to the choice of training data: (i) taken from the VOC `trainval` data provided, or (ii) from any source excluding the VOC `test` data provided:

No.	Task	Training data	Test data
3	Detection	trainval	test
4	Detection	any but VOC test	test

In competition 3, any annotation provided in the **VOC train and val sets** may be used for training, for example **bounding boxes** or particular views e.g. ‘frontal’ or ‘left’. Participants are *not* permitted to perform additional manual annotation of either training or test data.

In competition 4, any source of training data may be used *except* the provided **test** images. Researchers who have pre-built systems trained on other data are particularly encouraged to participate. The test data includes images from “flickr” (www.flickr.com); this source of images may *not* be used for training. Participants who have acquired images from flickr for training must submit them to the organizers to check for overlap with the test set.

4.3 Submission of Results

A separate text file of results should be generated for each competition (3 or 4) and each class e.g. ‘car’. Each line should be a detection output by the detector in the following format:

`<image identifier> <confidence> <left> <top> <right> <bottom>`

where `(left,top)-(right,bottom)` defines the bounding box of the detected object. The top-left pixel in the image has coordinates (1,1). Greater confidence values signify greater confidence that the detection is correct. An example file excerpt is shown below. Note that for the image 2009_000032, multiple objects are detected:

`comp3_det_test_car.txt:`

```
...
2009_000026 0.949297 172.000000 233.000000 191.000000 248.000000
2009_000032 0.013737 1.000000 147.000000 114.000000 242.000000
2009_000032 0.013737 1.000000 134.000000 94.000000 168.000000
2009_000035 0.063948 455.000000 229.000000 491.000000 243.000000
...
```

The example detector implementation (section 9.2.2) includes code for generating a results file in the required format.

4.4 Evaluation

The detection task will be judged by the precision/recall curve. The principal quantitative measure used will be the average precision (AP) (see section 3.4.1). Example code for computing the precision/recall and AP measure is provided in the development kit.

Detections are considered true or false positives based on the area of overlap with ground truth bounding boxes. To be considered a correct detection, the area of overlap a_o between the predicted bounding box B_p and ground truth bounding box B_{gt} must exceed 50% by the formula:

$$a_o = \frac{\text{area}(B_p \cap B_{gt})}{\text{area}(B_p \cup B_{gt})} \quad (1)$$

Example code for computing this overlap measure is provided in the development kit. Multiple detections of the *same* object in an image are considered *false* detections e.g. 5 detections of a single object is counted as 1 correct detection and 4 false detections – it is the responsibility of the participant’s system to filter multiple detections from its output.

Objects marked as ‘difficult’ (section 2.5) are currently *ignored* by the evaluation. The final evaluation may include separate results including such “difficult” images, depending on the submitted results.

Participants are expected to submit a *single* set of results per method employed. Participants who have investigated several algorithms may submit one result per method. Changes in algorithm parameters do *not* constitute a different method – all parameter tuning must be conducted using the training and validation data alone.

5 Segmentation Task

5.1 Task

For each test image pixel, predict the class of the object containing that pixel or ‘background’ if the pixel does not belong to one of the twenty specified classes. The output from your system should be an indexed image with each pixel index indicating the number of the inferred class (1-20) or zero, indicating background.

5.2 Competitions

Two competitions are defined according to the choice of training data: (i) taken from the VOC **trainval** data provided, or (ii) from any source excluding the VOC **test** data provided:

No.	Task	Training data	Test data
5	Segmentation	trainval	test
6	Segmentation	any but VOC test	test

For competition 5, any annotation provided in the VOC **train** and **val** sets may be used for training, for example segmentation, bounding boxes or particular views e.g. ‘frontal’ or ‘left’. However, if training uses annotation of any images other than the segmented training images, this must be reported as part of the submission (see below) since this allows a considerably larger training set. Participants are *not* permitted to perform additional manual annotation of either training or test data.

For competition 6, any source of training data may be used *except* the provided **test** images.

5.3 Submission of Results

Submission of results should be as collections of PNG format indexed image files, one per test image, with pixel indices from 0 to 20. The PNG color map should be the same as the color map used in the provided training and validation annotation (MATLAB users can use `VOClabelcolormap` – see section 10.5.3). The example segmenter implementation (section 9.2.3) includes code for generating

results in the required format. Participants may choose to include segmentations for only a subset of the 20 classes in which case they will be evaluated on only the included classes.

For competition 5, along with the submitted image files, participants **must** also state whether their method used segmentation training data only or both segmentation and bounding box training data. This information will be used when analysing and presenting the competition results.

5.4 Evaluation

Each segmentation competition will be judged by average segmentation accuracy across the twenty classes and the background class. The segmentation accuracy for a class will be assessed using the intersection/union metric, defined as the number of correctly labelled pixels of that class, divided by the number of pixels labelled with that class in either the ground truth labelling or the inferred labelling. Equivalently, the accuracy is given by the equation,

$$\text{segmentation accuracy} = \frac{\text{true positives}}{\text{true positives} + \text{false positives} + \text{false negatives}}$$

Code is provided to compute segmentation accuracies for each class, and the overall average accuracy (see section 10.5.2).

Participants are expected to submit a *single* set of results per method employed. Participants who have investigated several algorithms may submit one result per method. Changes in algorithm parameters do *not* constitute a different method – all parameter tuning must be conducted using the training and validation data alone.

6 Action Classification Task

6.1 Task

For each of the ten action classes predict if a specified person (indicated by their bounding box) in a test image is performing the corresponding action. The output from your system should be a real-valued confidence that the action is being performed so that a precision/recall curve can be drawn. Participants may choose to tackle all, or any subset of action classes, for example “walking only” or “walking and running”. Note that instances of the ‘other’ action class are included in the training/test sets as negative examples; a classifier targeting the ‘other’ action class is not required.

6.2 Competitions

Two competitions are defined according to the choice of training data: (i) taken from the VOC **trainval** data provided, or (ii) from any source excluding the VOC **test** data provided:

No.	Task	Training data	Test data
9	Action Classification	trainval	test
10	Action Classification	any but VOC test	test

In competition 9, any annotation provided in the VOC `train` and `val` sets may be used for training. Participants may use images and annotation for any of the competitions for training e.g. horse bounding boxes/segmentation to learn ‘ridinghorse’. Participants are *not* permitted to perform additional manual annotation of either training or test data.

In competition 10, any source of training data may be used *except* the provided `test` images. Researchers who have pre-built systems trained on other data are particularly encouraged to participate. The test data includes images from “flickr” (www.flickr.com); this source of images may *not* be used for training. Participants who have acquired images from flickr for training must submit them to the organizers to check for overlap with the test set.

6.3 Submission of Results

A separate text file of results should be generated for each competition (9 or 10) and each action class e.g. ‘phoning’. Each line should contain a single image identifier, object index, and the confidence output by the classifier, separated by a space, for example:

```
comp9_action_test_phoning.txt:
...
2010_006107 1 0.241236
2010_006107 2 0.758739
2010_006108 1 0.125374
...
```

The image identifier and object index specify the ‘person’ object to which the output corresponds; these are provided in the corresponding image sets. Greater confidence values signify greater confidence that the person is performing the action of interest. The example implementation (section 9.2.4) includes code for generating a results file in the required format.

6.4 Evaluation

The action classification task will be judged by the precision/recall curve. The principal quantitative measure used will be the average precision (AP) (see section 3.4.1). Example code for computing the precision/recall and AP measure is provided in the development kit.

Participants are expected to submit a *single* set of results per method employed. Participants who have investigated several algorithms may submit one result per method. Changes in algorithm parameters do *not* constitute a different method – all parameter tuning must be conducted using the training and validation data alone.

7 Boxless Action Classification Taster

7.1 Task

The boxless action classification task is the same as the action classification task, except that the person to be classified in a test image is indicated *only* by

a single point lying somewhere on their body, instead of a tight bounding box. The aim is to evaluate the efficacy of action classification methods when they are not provided with “precise” information about the extent of the person (in the form a tight bounding box), as might be the case where the input to the method is obtained from a generic human detector.

At *training* time, participants may use any of the annotation provided in the VOC dataset e.g. both reference point and bounding box for a person. At *test* time, a method must *only* use the reference point provided to identify the person to be classified – the bounding box present in the test image annotation must *not* be used.

7.2 Competitions

Two competitions are defined according to the choice of training data: (i) taken from the VOC **trainval** data provided, or (ii) from any source excluding the VOC **test** data provided:

No.	Task	Training data	Test data
11	Action Classification	trainval	test (point only)
12	Action Classification	any but VOC test	test (point only)

In competition 11, any annotation provided in the VOC **train** and **val** sets may be used for training. Participants may use images and annotation for any of the competitions for training e.g. horse bounding boxes/segmentation to learn ‘ridinghorse’. Participants are *not* permitted to perform additional manual annotation of either training or test data.

In competition 12, any source of training data may be used *except* the provided **test** images. Researchers who have pre-built systems trained on other data are particularly encouraged to participate. The test data includes images from “flickr” (www.flickr.com); this source of images may *not* be used for training. Participants who have acquired images from flickr for training must submit them to the organizers to check for overlap with the test set.

Note that in *both* competitions the only annotation in the test images that may be used is the reference point for a person indicating which person is to be classified. It is *not* allowed to use the bounding boxes in the test annotation.

7.3 Submission of Results and Evaluation

Results should submitted in the same format as for the action classification task (using bounding boxes), using competition numbers 11 or 12 to identify that only reference points have been used for the test data. The taster will be evaluated in the same way as the action classification task.

8 Person Layout Taster

8.1 Task

For each ‘person’ object in a test image (their bounding box is provided) predict the presence/absence of parts (head/hands/feet), and the bounding boxes

of those parts. The prediction for a person layout should be output with an associated real-valued confidence of the layout so that a precision/recall curve can be drawn. Only a single estimate of layout should be output for each person.

The success of the layout prediction depends both on: (i) a correct prediction of parts present/absent (e.g. is the hand visible or occluded); (ii) a correct prediction of bounding boxes for the visible parts.

8.2 Competitions

Two competitions are defined according to the choice of training data: (i) taken from the VOC **trainval** data provided, or (ii) from any source excluding the VOC **test** data provided:

No.	Task	Training data	Test data
7	Layout	trainval	test
8	Layout	any but VOC test	test

In competition 7, any annotation provided in the VOC **train** and **val** sets may be used for training, for example bounding boxes or particular views e.g. ‘frontal’ or ‘left’. Participants are *not* permitted to perform additional manual annotation of either training or test data.

In competition 8, any source of training data may be used *except* the provided **test** images. Researchers who have pre-built systems trained on other data are particularly encouraged to participate. The test data includes images from “flickr” (www.flickr.com); this source of images may *not* be used for training. Participants who have acquired images from flickr for training must submit them to the organizers to check for overlap with the test set.

8.3 Submission of Results

To support the hierarchical (person+parts) nature of this task, an XML format has been adopted for submission of results. A separate XML file of results should be generated for each competition (6 or 7). The overall format should follow:

```
<results>
  <layout>
    ... layout estimate 1 ...
  </layout>
  <layout>
    ... layout estimate 2 ...
  </layout>
</results>
```

Each detection is represented by a **<layout>** element. The order of detections is not important. An example detection is shown here:

```
<layout>
  <image>2009_000183</image>
  <object>1</object>
  <confidence>-1189</confidence>
  <part>
```

```

        <class>head</class>
        <bndbox>
            <xmin>191</xmin>
            <ymin>25</ymin>
            <xmax>323</xmax>
            <ymax>209</ymax>
        </bndbox>
    </part>
    <part>
        <class>hand</class>
        <bndbox>
            <xmin>393</xmin>
            <ymin>206</ymin>
            <xmax>488</xmax>
            <ymax>300</ymax>
        </bndbox>
    </part>
    <part>
        <class>hand</class>
        <bndbox>
            <xmin>1</xmin>
            <ymin>148</ymin>
            <xmax>132</xmax>
            <ymax>329</ymax>
        </bndbox>
    </part>
</layout>

```

The `<image>` element specifies the image identifier. The `<object>` specifies the index of the object to which the layout relates (the first object in the image has index 1) and should match that provided in the image set files (section 10.1.6). The `<confidence>` element specifies the confidence of the layout estimate, used to generate a precision/recall curve as in the detection task.

Each `<part>` element specifies the detection of a particular part of the person i.e. head/hand/foot. If the part is predicted to be absent/invisible, the corresponding element should be omitted. For each part, the `<class>` element specifies the type of part: `head`, `hand` or `foot`. The `<bndbox>` element specifies the predicted bounding box for that part; bounding boxes are specified in image co-ordinates and need not be contained in the provided person bounding box.

To ease creation of the required XML results file for MATLAB users, a function is included in the development kit to convert MATLAB structures to XML. See the `VOCwritexml` function (section 10.7.1). The example person layout implementation (section 9.2.6) includes code for generating a results file in the required format.

8.4 Evaluation

The person layout task will principally be judged by how well each part *individually* can be predicted. For each of the part types (head/hands/feet) a precision/recall curve will be computed, using the confidence supplied with the

person layout to determine the ranking. A prediction of a part is considered true or false according to the overlap test, as used in the detection challenge, i.e. for a true prediction the bounding box of the part overlaps the ground truth by at least 50%. For each part type, the principal quantitative measure used will be the average precision (AP) (see section 3.4.1). Example code for computing the precision/recall curves and AP measure is provided in the development kit.

Invitation to propose evaluation schemes. We invite participants to propose additional evaluation schemes for the person layout task. In particular, we are interested in schemes which (i) evaluate accuracy of *complete* layout predictions; (ii) incorporate the notion of ranking of results by confidence. If you have a successful layout prediction method and insight please propose promising evaluation techniques in the form of (i) motivation and explanation; (ii) MATLAB implementation compatible with the VOC results format.

9 Development Kit

The development kit is packaged in a single gzipped tar file containing MATLAB code and (this) documentation. The images, annotation, and lists specifying training/validation sets for the challenge are provided in a separate archive which can be obtained via the VOC web pages [1].

9.1 Installation and Configuration

The simplest installation is achieved by placing the development kit and challenge databases in a single location. After untarring the development kit, download the challenge image database and untar into the same directory, resulting in the following directory structure:

```
VOCdevkit/                % development kit
VOCdevkit/VOCcode/        % VOC utility code
VOCdevkit/results/VOC2012/ % your results on VOC2012
VOCdevkit/local/          % example code temp dirs
VOCdevkit/VOC2012/ImageSets % image sets
VOCdevkit/VOC2012/Annotations % annotation files
VOCdevkit/VOC2012/JPEGImages % images
VOCdevkit/VOC2012/SegmentationObject % segmentations by object
VOCdevkit/VOC2012/SegmentationClass % segmentations by class
```

If you set the current directory in MATLAB to the VOCdevkit directory you should be able to run the example functions:

- `example_classifier`
- `example_detector`
- `example_segmenter`
- `example_action`
- `example_action_nobb`

- `example_layout`

If desired, you can store the code, images/annotation, and results in separate directories, for example you might want to store the image data in a common group location. To specify the locations of the image/annotation, results, and working directories, edit the `VOCinit.m` file, e.g.

```
% change this path to point to your copy of the PASCAL VOC data
VOCopts.datadir='/homes/group/VOCdata/';
```

```
% change this path to a writable directory for your results
VOCopts.resdir='/homes/me/VOCresults/';
```

```
% change this path to a writable local directory for the example code
VOCopts.localdir='/tmp/';
```

Note that in developing your own code you need to include the `VOCdevkit/VOCcode` directory in your MATLAB path, e.g.

```
>> addpath /homes/me/code/VOCdevkit/VOCcode
```

9.2 Example Code

Example implementations are provided for all tasks. The aim of these (minimal) implementations is solely to demonstrate use of the code in the development kit.

9.2.1 Example Classifier Implementation

The file `example_classifier.m` contains a complete implementation of the classification task. For each VOC object class a simple classifier is trained on the `train` set; the classifier is then applied to the `val` set and the output saved to a results file in the format required by the challenge; a precision/recall curve is plotted and the ‘average precision’ (AP) measure displayed.

9.2.2 Example Detector Implementation

The file `example_detector.m` contains a complete implementation of the detection task. For each VOC object class a simple (and not very successful!) detector is trained on the `train` set; the detector is then applied to the `val` set and the output saved to a results file in the format required by the challenge; a precision/recall curve is plotted and the ‘average precision’ (AP) measure displayed.

9.2.3 Example Segmenter Implementation

An example segmenter is provided which converts detection results into segmentation results, using `create_segmentations_from_detections` (described below). For example:

```
>> example_detector;
>> example_segmenter;
```

This runs the example detector, converts the detections into segmentations and displays a table of per-class segmentation accuracies, along with an overall average accuracy.

9.2.4 Example Action Implementation

The file `example_action.m` contains a complete implementation of the action classification task. For each VOC action class a simple classifier is trained on the `train` set; the classifier is then applied to all specified ‘person’ objects in the `val` set and the output saved to a results file in the format required by the challenge; a precision/recall curve is plotted and the ‘average precision’ (AP) measure displayed.

9.2.5 Example Boxless Action Implementation

The file `example_action_nobb.m` contains a complete implementation of the boxless action classification task, using only the reference point annotation of people in the test images (no bounding boxes). For each VOC action class a simple classifier is trained on the `train` set; the classifier is then applied to all specified ‘person’ objects in the `val` set and the output saved to a results file in the format required by the challenge; a precision/recall curve is plotted and the ‘average precision’ (AP) measure displayed.

9.2.6 Example Layout Implementation

The file `example_layout.m` contains a complete implementation of the person layout task. A simple (and not very successful!) layout predictor is trained on the `train` set; the layout predictor is then applied to the `val` set and the output saved to a results file in the format required by the challenge; a precision/recall curve is plotted and the ‘average precision’ (AP) measure displayed.

9.3 Non-MATLAB Users

For non-MATLAB users, the file formats used for the VOC2012 data should be straightforward to use in other environments. Image sets (see below) are vanilla text files. Annotation files are XML format and should be readable by any standard XML parser. Images are stored in JPEG format, and segmentation ground truth in PNG format.

10 Using the Development Kit

The development kit provides functions for loading annotation data. Example code for computing precision/recall curves and segmentation accuracy, and for viewing annotation is also provided.

10.1 Image Sets

10.1.1 Classification/Detection Task Image Sets

The `VOC2012/ImageSets/Main/` directory contains text files specifying lists of images for the main classification/detection tasks.

The files `train.txt`, `val.txt`, `trainval.txt` and `test.txt` list the image identifiers for the corresponding image sets (training, validation, training+validation and testing). Each line of the file contains a single image iden-

tifier. The following MATLAB code reads the image list into a cell array of strings:

```
imgset='train';  
ids=textread(sprintf(VOCopts.imgsetpath,imgset),'%s');
```

For a given image identifier `ids{i}`, the corresponding image and annotation file paths can be produced thus:

```
imgpath=sprintf(VOCopts.imgpath,ids{i});  
annopath=sprintf(VOCopts.annopath,ids{i});
```

Note that the image sets used are the same for all classes. For each competition, participants are expected to provide output for all images in the `test` set.

10.1.2 Classification Task Image Sets

To simplify matters for participants tackling only the *classification* task, class-specific image sets with per-image ground truth are also provided. The file `VOC2012/ImageSets/Main/<class>_<imgset>.txt` contains image identifiers and ground truth for a particular class and image set, for example the file `car_train.txt` applies to the ‘car’ class and `train` image set.

Each line of the file contains a single image identifier and ground truth label, separated by a space, for example:

```
...  
2009_000040 -1  
2009_000042 -1  
2009_000052 1  
...
```

The following MATLAB code reads the image list into a cell array of strings and the ground truth label into a corresponding vector:

```
imgset='train';  
cls='car';  
[ids,gt]=textread(sprintf(VOCopts.clsimgsetpath, ...  
                           cls,imgset),'%s %d');
```

There are *three* ground truth labels:

- 1: Negative: The image contains no objects of the class of interest. A classifier should give a ‘negative’ output.
- 1: Positive: The image contains at least one object of the class of interest. A classifier should give a ‘positive’ output.
- 0: “Difficult”: The image contains only objects of the class of interest marked as ‘difficult’.

The “difficult” label indicates that all objects of the class of interest have been annotated as “difficult”, for example an object which is clearly visible but difficult to recognize without substantial use of context. Currently the evaluation ignores such images, contributing nothing to the precision/recall curve or AP measure. The final evaluation may include separate results including such “difficult” images, depending on the submitted results. Participants are free to omit these images from training or include as either positive or negative examples.

10.1.3 Segmentation Image Sets

The `VOC2012/ImageSets/Segmentation/` directory contains text files specifying lists of images for the segmentation task.

The files `train.txt`, `val.txt`, `trainval.txt` and `test.txt` list the image identifiers for the corresponding image sets (training, validation, training+validation and testing). Each line of the file contains a single image identifier. The following MATLAB code reads the image list into a cell array of strings:

```
imgset='train';  
ids=textread(sprintf(VOCopts.seg.imgsetpath,imgset),'%s');
```

For a given image identifier `ids{i}`, file paths for the corresponding image, annotation, segmentation by object instance and segmentation by class can be produced thus:

```
imgpath=sprintf(VOCopts.imgpath,ids{i});  
annopath=sprintf(VOCopts.annopath,ids{i});  
clsseghpath=sprintf(VOCopts.seg.clsimgpath,ids{i});  
objseghpath=sprintf(VOCopts.seg.instimgpath,ids{i});
```

Participants are expected to provide output for all images in the `test` set.

10.1.4 Action Classification Image Sets

The `VOC2012/ImageSets/Action/` directory contains text files specifying lists of images and ‘person’ objects for the action classification tasks.

The files `train.txt`, `val.txt`, `trainval.txt` and `test.txt` list the image identifiers for the corresponding image sets (training, validation, training+validation and testing). Each line of the file contains a single image identifier. The following MATLAB code reads the image list into a cell array of strings:

```
imgset='train';  
ids=textread(sprintf(VOCopts.action.imgsetpath,imgset),'%s');
```

For a given image identifier `ids{i}`, the corresponding image and annotation file paths can be produced thus:

```
imgpath=sprintf(VOCopts.imgpath,ids{i});  
annopath=sprintf(VOCopts.annopath,ids{i});
```

Note that the image sets used are the same for all action classes. For each competition, participants are expected to provide output for all ‘person’ objects in each image of the `test` set.

10.1.5 Action Class Image Sets

To simplify matters for participants tackling the action classification task, action class-specific image sets with per-object ground truth are also provided. The file `VOC2012/ImageSets/Action/<class>_<imgset>.txt` contains image identifiers, object indices and ground truth for a particular action class and image set, for example the file `phoning.train.txt` applies to the ‘phoning’ action class and `train` image set.

Each line of the file contains a single image identifier, single object index, and ground truth label, separated by a space, for example:

```
...
2010_006215  1  1
2010_006217  1 -1
2010_006217  2 -1
...
```

The following MATLAB code reads the image identifiers into a cell array of strings, the object indices into a vector, and the ground truth label into a corresponding vector:

```
imgset='train'; cls='phoning';
[imgids,objids,gt]=textread(sprintf(VOCopts.action.clsimgsetpath,
                                     cls,imgset),'%s %d %d');
```

The annotation for the object (bounding box, reference point and actions) can then be obtained using the image identifier and object index:

```
rec=PASreadrecord(sprintf(VOCopts.annopath,imgids{i}));
obj=rec.objects(objids{i});
```

There are two ground truth labels:

- 1: Negative: The person is not performing the action of interest. A classifier should give a 'negative' output.
- 1: Positive: The person is performing the action of interest. A classifier should give a 'positive' output.

10.1.6 Person Layout Taster Image Sets

The `VOC2012/ImageSets/Layout/` directory contains text files specifying lists of image for the person layout taster task.

The files `train.txt`, `val.txt`, `trainval.txt` and `test.txt` list the image identifiers for the corresponding image sets (training, validation, training+validation and testing). Each line of the file contains a single image identifier, and a single object index. Together these specify a 'person' object for which layout is provided or to be estimated, for example:

```
...
2009_000595  1
2009_000595  2
2009_000606  1
...
```

The following MATLAB code reads the image list into a cell array of strings and the object indices into a corresponding vector:

```
imgset='train';
[imgids,objids]=textread(sprintf(VOCopts.layout.imgsetpath, ...
                                 VOCopts.trainset),'%s %d');
```

The annotation for the object (bounding box only in the test data) can then be obtained using the image identifier and object index:

```
rec=PASreadrecord(sprintf(VOCopts.annopath,imgids{i}));
obj=rec.objects(objids{i});
```

10.2 Development Kit Functions

10.2.1 VOCinit

The `VOCinit` script initializes a single structure `VOCopts` which contains options for the PASCAL functions including directories containing the VOC data and options for the evaluation functions (not to be modified).

The field `classes` lists the object classes for the challenge in a cell array:

```
VOCopts.classes={'aeroplane','bicycle','bird','boat',...
                'bottle','bus','car','cat',...
                'chair','cow','diningtable','dog',...
                'horse','motorbike','person','pottedplant',...
                'sheep','sofa','train','tvmonitor'};
```

The field `actions` lists the action classes for the action classification task in a cell array:

```
VOCopts.actions={'other','jumping','phoning','playinginstrument',...
                'reading','ridingbike','ridinghorse','running',...
                'takingphoto','usingcomputer','walking'};
```

The field `trainset` specifies the image set used by the example evaluation functions for training:

```
VOCopts.trainset='train'; % use train for development
```

Note that participants are free to use both training and validation data in any manner they choose for the final challenge.

The field `testset` specifies the image set used by the example evaluation functions for testing:

```
VOCopts.testset='val'; % use validation data for development
```

Other fields provide, for convenience, paths for the image and annotation data and results files. The use of these paths is illustrated in the example implementations.

10.2.2 PASreadrecord(filename)

The `PASreadrecord` function reads the annotation data for a particular image from the annotation file specified by `filename`, for example:

```
>> rec=PASreadrecord(sprintf(VOCopts.annopath,'2009_000067'))
```

```
rec =
```

```
    folder: 'VOC2009'
```

```

filename: '2009_000067.jpg'
source: [1x1 struct]
size: [1x1 struct]
segmented: 0
imgname: 'VOC2009/JPEGImages/2009_000067.jpg'
imgsize: [500 334 3]
database: 'The VOC2009 Database'
objects: [1x6 struct]

```

The `imgname` field specifies the path (relative to the main VOC data path) of the corresponding image. The `imgsize` field specifies the image dimensions as (width,height,depth). The `database` field specifies the data source (e.g. VOC2009 or VOC2012). The `segmented` field specifies if a segmentation is available for this image. The `folder` and `filename` fields provide an alternative specification of the image path, and `size` an alternative specification of the image size:

```
>> rec.size
```

```
ans =
```

```

width: 500
height: 334
depth: 3

```

The `source` field contains additional information about the source of the image e.g. web-site and owner. This information is obscured until completion of the challenge.

Objects annotated in the image are stored in the struct array `objects`, for example:

```
>> rec.objects(2)
```

```
ans =
```

```

class: 'person'
view: 'Right'
truncated: 0
occluded: 0
difficult: 0
label: 'PAspersonRight'
orglabel: 'PAspersonRight'
bbox: [225 140 270 308]
bndbox: [1x1 struct]
polygon: []
mask: []
hasparts: 1
part: [1x4 struct]

```

The `class` field contains the object class. The `view` field contains the view: Frontal, Rear, Left (side view, facing left of image), Right (side view, facing right of image), or an empty string indicating another, or un-annotated view.

The `truncated` field being set to 1 indicates that the object is “truncated” in the image. The definition of truncated is that the bounding box of the object specified does not correspond to the full extent of the object e.g. an image of a person from the waist up, or a view of a car extending outside the image. Participants are free to use or ignore this field as they see fit.

The `occluded` field being set to 1 indicates that the object is significantly occluded by another object. Participants are free to use or ignore this field as they see fit.

The `difficult` field being set to 1 indicates that the object has been annotated as “difficult”, for example an object which is clearly visible but difficult to recognize without substantial use of context. Currently the evaluation ignores such objects, contributing nothing to the precision/recall curve. The final evaluation may include separate results including such “difficult” objects, depending on the submitted results. Participants may include or exclude these objects from training as they see fit.

The `bbox` field specifies the bounding box of the object in the image, as `[left,top,right,bottom]`. The top-left pixel in the image has coordinates (1,1). The `bndbox` field specifies the bounding box in an alternate form:

```
>> rec.objects(2).bndbox
```

```
ans =
```

```
    xmin: 225
    ymin: 140
    xmax: 270
    ymax: 308
```

For backward compatibility, the `label` and `orglabel` fields specify the PASCAL label for the object, comprised of class, view and truncated/difficult flags. The `polygon` and `mask` fields specify polygon/per-object segmentations, and are not provided for the VOC2012 data.

The `hasparts` field specifies if the object has sub-object “parts” annotated. For the VOC2012 data, such annotation is available for a subset of the ‘person’ objects, used in the layout taster task. Object parts are stored in the struct array `part`, for example:

```
>> rec.objects(2).part(1)
```

```
ans =
```

```
    class: 'head'
    view: ''
    truncated: 0
    occluded: 0
    difficult: 0
    label: 'PAShead'
    orglabel: 'PAShead'
    bbox: [234 138 257 164]
    bndbox: [1x1 struct]
    polygon: []
```

```

        mask: []
    hasparts: 0
    part: []

```

The format of object parts is identical to that for top-level objects. For the ‘person’ parts in the VOC2012 data, parts are not annotated with view, or truncated/difficult flags. The bounding box of a part is specified in image coordinates in the same way as for top-level objects. Note that the object parts may legitimately extend outside the bounding box of the parent object.

For ‘person’ objects in the action classification image sets, objects are additionally annotated with the set of actions being performed and a reference point on the person’s body. The `hasactions` field specifies if the object has actions annotated. Action flags are stored in the struct `actions`, for example:

```
>> rec.objects(1).actions
```

```
ans =
```

```

        other: 0
        phoning: 1
    playinginstrument: 0
        reading: 0
        ridingbike: 0
        ridinghorse: 0
        running: 0
        takingphoto: 0
        usingcomputer: 0
        walking: 0

```

There is one flag for each of the ten action classes plus ‘other’, with the flag set to true (1) if the person is performing the corresponding action. Note that actions except ‘other’ are not mutually-exclusive.

Each person in the action classification image sets is additionally annotated with a reference point which is used to indicate the person’s approximate location for the boxless action classification task. The `haspoint` field specifies if the object has a reference point annotated, which is stored in the struct `point`, for example:

```
>> rec.objects(1).point
```

```
ans =
```

```

    x: 186
    y: 210

```

The point is guaranteed to lie on the body and to be unoccluded by other objects. Typically the point is located around the middle of their chest, although this varies depending on pose and occlusions.

10.2.3 viewanno(imgset)

The `viewanno` function displays the annotation for images in the image set specified by `imgset`. Some examples:

```
>> viewanno('Main/train');
>> viewanno('Main/car_val');
>> viewanno('Layout/train');
>> viewanno('Segmentation/val');
>> viewanno('Action/trainval');
```

10.3 Classification Functions

10.3.1 V0Cevalcls(V0Copts,id,cls,draw)

The `V0Cevalcls` function performs evaluation of the classification task, computing a precision/recall curve and the average precision (AP) measure. The arguments `id` and `cls` specify the results file to be loaded, for example:

```
>> [rec,prec,ap]=V0Cevalcls(V0Copts,'comp1','car',true);
```

See `example_classifier` for further examples. If the argument `draw` is true, the precision/recall curve is drawn in a figure window. The function returns vectors of recall and precision rates in `rec` and `prec`, and the average precision measure in `ap`.

10.4 Detection Functions

10.4.1 V0Cevaldet(V0Copts,id,cls,draw)

The `V0Cevaldet` function performs evaluation of the detection task, computing a precision/recall curve and the average precision (AP) measure. The arguments `id` and `cls` specify the results file to be loaded, for example:

```
>> [rec,prec,ap]=V0Cevaldet(V0Copts,'comp3','car',true);
```

See `example_detector` for further examples. If the argument `draw` is true, the precision/recall curve is drawn in a figure window. The function returns vectors of recall and precision rates in `rec` and `prec`, and the average precision measure in `ap`.

10.4.2 viewdet(id,cls,onlytp)

The `viewdet` function displays the detections stored in a results file for the detection task. The arguments `id` and `cls` specify the results file to be loaded, for example:

```
>> viewdet('comp3','car',true)
```

If the `onlytp` argument is true, only the detections considered true positives by the VOC evaluation measure are displayed.

10.5 Segmentation Functions

10.5.1 create_segmentations_from_detections(id,confidence)

This function creates segmentation results from detection results.

`create_segmentations_from_detections(id)` creates segmentations from the detection results with specified identifier e.g. `comp3`. This is achieved by

rendering the bounding box for each detection in class order, so that later classes overwrite earlier classes (e.g. a person bounding box will overwrite an overlapping an aeroplane bounding box). All detections will be used, no matter what their confidence level.

`create_segmentations_from_detections(id,confidence)` does the same, but only detections above the specified confidence will be used.

See `example_segmenter` for an example.

10.5.2 `VOCevalseg(VOCopts,id)`

The `VOCevalseg` function performs evaluation of the segmentation task, computing a confusion matrix and segmentation accuracies for the segmentation task. It returns per-class percentage accuracies, the average overall percentage accuracy, and a confusion matrix, for example:

```
>> [accuracies,avacc,conf,rawcounts] = VOCevalseg(VOCopts,'comp3')
```

Accuracies are defined by the intersection/union measure. The optional fourth output 'rawcounts' returns an un-normalized confusion matrix containing raw pixel counts. See `example_segmenter` for another example. This function will also display a table of overall and per-class accuracies.

10.5.3 `VOClabelcolormap(N)`

The `VOClabelcolormap` function creates the color map which has been used for all provided indexed images. You should use this color map for writing your own indexed images, for consistency. The size of the color map is given by `N`, which should generally be set to 256 to include a color for the 'void' label.

10.6 Action Functions

10.6.1 `VOCevalaction(VOCopts,id,cls,draw)`

The `VOCevalaction` function performs evaluation of the action classification task, computing a precision/recall curve and the average precision (AP) measure. The arguments `id` and `cls` specify the results file to be loaded, for example:

```
>> [rec,prec,ap]=VOCevalcls(VOCopts,'comp9','phoning',true);
```

See `example_action` for further examples. If the argument `draw` is true, the precision/recall curve is drawn in a figure window. The function returns vectors of recall and precision rates in `rec` and `prec`, and the average precision measure in `ap`.

Note that the same evaluation function applies to both the action classification task and boxless action classification task.

10.7 Layout Functions

10.7.1 `VOCwritexml(rec,path)`

The `VOCwritexml` function writes a MATLAB structure array to a corresponding XML file. It is provided to support the creation of XML results files for the person layout taster. An example of usage can be found in `example_layout`.

10.7.2 VOCevallayout_pr(VOCopts,id,draw)

The `VOCevallayout_pr` function performs evaluation of the person layout task, computing a precision/recall curve and the average precision (AP) measure for each part type (head/hands/feet). The arguments `id` and `cls` specify the results file to be loaded, for example:

```
>> [rec,prec,ap]=VOCevallayout_pr(VOCopts,'comp7',true);
```

See `example_layout` for further examples. If the argument `draw` is true, the precision/recall curves are drawn in a figure window. The function returns vectors of recall and precision rates in `reci` and `prec{i}`, and the average precision measure in `ap{i}`, where the index `i` indexes the part type in `VOCopts.parts`.

Acknowledgements

We gratefully acknowledge the following, who spent many long hours providing annotation for the VOC2012 database: Yusuf Aytar, Lucia Ballerini, Hakan Bilen, Ken Chatfield, Mircea Cimpoi, Ali Eslami, Basura Fernando, Christoph Godau, Bertan Gunyel, Phoenix/Xuan Huang, Jyri Kivinen, Markus Mathias, Kristof Overdulse, Konstantinos Rematas, Johan Van Rompay, Gilad Sharir, Mathias Vercruysse, Vibhav Vineet, Ziming Zhang, Shuai Kyle Zheng.

The preparation and running of this challenge is supported by the EU-funded PASCAL2 Network of Excellence on Pattern Analysis, Statistical Modelling and Computational Learning.

References

- [1] The PASCAL Visual Object Classes Challenge (VOC2012). <http://pascallin.ecs.soton.ac.uk/challenges/VOC/voc2012/index.html>.